1. pivot_table

Single aggregation

```
pd.pivot_table(df,
        values='column',
        index='row_column',
        aggfunc='mean')
```

Multiple aggregations on one column

```
pd.pivot_table(df,
        values='column',
        index='row_column',
        aggfunc=['mean','sum','median'])
```

Different aggregation for different columns

```
pd.pivot_table(df,
        values=['col1','col2'],
        index='row_column',
        aggfunc={
          'col1':'mean',
          'col2':'sum'
        })
```

Multiple aggregations for multiple columns

```
pd.pivot_table(df,
        values=['col1','col2'],
        index='row_column',
        aggfunc={
          'col1':['mean','median'],
          'col2':['sum','max']
        })
```

Using a custom (def) function

```
def my_function(x):
  return ...

pd.pivot_table(df,
        values='column',
        index='row_column',
        aggfunc=my_function)
```

or

```
pd.pivot_table(df,
        values='column',
        index='row_column',
        aggfunc=[my_function, 'mean'])
```

These patterns cover the vast majority of pivot_table() use cases in pandas

---

2. PD.MERGE

Goal	Correct Code
Merge on id	pd.merge(df1, df2, on='id', how='left')
Merge on id and name	pd.merge(df1, df2, on=['id', 'name'], how='left')
Different column names	pd.merge(df1, df2, left_on='id', right_on='emp_id', how='left')

---

3. GROUP BY

General Syntax Summary

One grouping column

```
df.groupby('Group_Column')['Value_Column'].sum()
```

Multiple grouping columns

```
df.groupby(['Group1', 'Group2'])['Value_Column'].sum()
```

Multiple aggregation functions

```
df.groupby(['Group1', 'Group2'])['Value_Column'].agg(
  ['mean', 'median', 'sum']
)
```

User-defined function

```
def my_function(x):
  return ...

df.groupby(['Group1', 'Group2'])['Value_Column'].agg(my_function)
```

Multiple user-defined and built-in functions together

```
def my_range(x):
  return x.max() - x.min()

df.groupby(['Group1', 'Group2'])['Value_Column'].agg(
  ['mean', 'median', 'sum', my_range]
)
```

---

4. CONCAT

Create DataFrames

```
import pandas as pd


df1 = pd.DataFrame({
  'ID': [1, 2],
  'Name': ['A', 'B']
})
```

```
df2 = pd.DataFrame({
  'ID': [3, 4],
  'Name': ['C', 'D']
})
```

df1

	ID	Name
0	1	A
1	2	B

df2

	ID	Name
0	3	C
1	4	D

---

1. Concatenate on axis=0 (Rows)

Code

```
pd.concat([df1, df2], axis=0)
```

Output

	ID	Name
0	1	A
1	2	B
0	3	C
1	4	D

---

2. Concatenate on axis=1 (Columns)

Code

```
pd.concat([df1, df2], axis=1)
```

Output

	ID	Name	ID	Name
0	1	A	3	C
1	2	B	4	D

---

3. Concatenate Using join='inner'

Make DataFrames with different indexes.

```
df1 = pd.DataFrame({
  'ID':[1,2],
  'Name':['A','B']
}, index=[0,1])


df2 = pd.DataFrame({
  'Age':[20,21]
}, index=[1,2])]
 # Change indexes
df1.index = [0, 1]
df2.index = [1, 2]


pd.concat([df1, df2], axis=1, join='inner')
```

Output

	ID	Name	Age
1	B		20

---

4. Concatenate Using join='outer'

Code

```
pd.concat([df1, df2], axis=1, join='outer')
```

Output

	ID	Name	Age
0	1	A	NaN
1	2	B	20
2	NaN	NaN	21

---

5. Concatenate with Keys (F, S)

Code

```
pd.concat([df1, df2], keys=['F', 'S'])
```

Output

		ID	Name
F	0	1	A
F	1	2	B
S	0	3	C
S	1	4	D

---

6. Concatenate on axis=1 with Keys (L, R)

Code

```
pd.concat([df1, df2], axis=1, keys=['L', 'R'])
```

Output

	L		R	
	ID	Name	ID	Name
0	1	A	3	C
1	2	B	4	D

---

5. EXPLODE

If you mean **explode() in Pandas**, here are 5 common use cases.

Case 1: Explode a Single List Column

```
import pandas as pd

df = pd.DataFrame({
  'Name': ['A', 'B'],
  'Subjects': [['Math', 'Physics'], ['Chemistry']]
})

df.explode('Subjects')
```

Output

Name	Subjects
A	Math
A	Physics
B	Chemistry

Case 2: Explode Multiple Columns Together

```
import pandas as pd

df = pd.DataFrame({
  'Name': ['A', 'B'],
  'Subject': [['Math', 'Physics'], ['Chemistry', 'Biology']],
  'Marks': [[90, 95], [88, 91]]
})

df.explode(['Subject', 'Marks'])
```

Output

Name	Subject	Marks
A	Math	90
A	Physics	95
B	Chemistry	88
B	Biology	91

Case 3: Explode a Column of Tuples

```
import pandas as pd

df = pd.DataFrame({
  'ID': [1, 2],
  'Location': [('Delhi', 'India'),
        ('Paris', 'France')]
})

df.explode('Location')
```

Output

ID	Location
1	Delhi
1	India
2	Paris
2	France

Case 4: Explode a Set

```
import pandas as pd
```

```
df = pd.DataFrame({
   'Student': ['A', 'B'],
   'Languages': [{'Python', 'SQL'},
            {'Java', 'C++'}]
})

df.explode('Languages')
```

Output *(order may vary because sets are unordered)*

Student	Languages
A	Python
A	SQL
B	Java
B	C++

Case 5: Convert a String to a List, Then Explode

```
import pandas as pd

df = pd.DataFrame({
   'Name': ['A', 'B'],
   'Skills': ['Python,SQL,Excel',
         'Java,C++']
})

df['Skills'] = df['Skills'].str.split(',')

df.explode('Skills')
```

Output

Name	Skills
A	Python
A	SQL
A	Excel
B	Java
B	C++

General Syntax

```
# Explode one column
df.explode('column_name')

# Explode multiple columns
df.explode(['col1', 'col2'], ignore_index=True)

# Reset index while exploding
df.explode('column_name', ignore_index=True)
```

explode() converts each element of a list-like object (list, tuple, set, etc.) in a cell into a separate row, while duplicating the values from the other columns.

---

6. INDEX

Operation	Code
1. Change Index	df.index = [101, 102, 103]
2. Set Index	df.set_index('ID')
2. Set Index (Modify Original)	df.set_index('ID', inplace=True)

2. Set Multiple Columns as Index	df.set_index(['ID', 'Name'])
3. Name the Index	df.index.name = 'Employee_ID'
4. Reset Index	df.reset_index()
4. Reset Index (Drop Old Index)	df.reset_index(drop=True)
4. Reset Index (Modify Original)	df.reset_index(inplace=True)
5. Sort Index (Ascending)	df.sort_index()
5. Sort Index (Descending)	df.sort_index(ascending=False)
6. Print/View Index	print(df.index)

---

7. SORT, FILTER, ILOC, LOC, UPDATE

1. Sort Values (sort_values())

Operation	Code
Sort by one column (Ascending)	df.sort_values(by='Salary')
Sort by one column (Descending)	df.sort_values(by='Salary', ascending=False)
Sort by multiple columns	df.sort_values(by=['Gender','Salary'])
Multiple columns (Different order)	df.sort_values(by=['Gender','Salary'], ascending=[True, False])
Modify original DataFrame	df.sort_values(by='Salary', inplace=True)

---

2. Sort Index (sort_index())

Operation	Code
Sort index (Ascending)	df.sort_index()
Sort index (Descending)	df.sort_index(ascending=False)
Modify original DataFrame	df.sort_index(inplace=True)

---

3. Filter Rows (Select required columns)

Operation	Code
Only columns Name & Salary	df[['Name','Salary']]
Only one column	df['Salary']
Multiple columns	df[['ID','Gender','Salary']]

---

4. Filter Rows Using Conditions (AND)

Operation	Code
Salary > 500	df[df['Salary'] > 500]
Gender == 'Female'	df[df['Gender'] == 'Female']
Salary > 500 AND Gender == 'Female'	df[(df['Salary'] > 500) & (df['Gender'] == 'Female')]
Age >= 25 AND Salary < 1000	df[(df['Age'] >= 25) & (df['Salary'] < 1000)]

---

5. Filter Using OR

Operation	Code
Salary > 500 OR Gender == 'Female'	df[(df['Salary'] > 500) | (df['Gender'] == 'Female')]
Age < 20 OR Salary > 1000	df[(df['Age'] < 20) | (df['Salary'] > 1000)]

---

6. iloc[] (Integer Position Based)

Operation	Code
First row	df.iloc[0]

Operation	Code
First row, first column	df.iloc[0,0]
First 3 rows	df.iloc[:3]
Rows 2–5	df.iloc[1:5]
First 3 rows, first 2 columns	df.iloc[:3,:2]
All rows, first 2 columns	df.iloc[:,:2]
Rows 2–4, columns 2–3	df.iloc[1:4,1:3]
Last row	df.iloc[-1]
Last column	df.iloc[:,-1]
Specific rows	df.iloc[[0,2,4]]
Specific columns	df.iloc[:,[0,2]]
Specific rows & columns	df.iloc[[0,2],[1,3]]

---

7. loc[] (Label Based)

Operation	Code
Row with index 101	df.loc[101]
Row 101, column Name	df.loc[101,'Name']
Rows 101–105	df.loc[101:105]
Rows 101–105, Name & Salary	df.loc[101:105,['Name','Salary']]
All rows, Name column	df.loc[:,'Name']
All rows, Name & Salary	df.loc[:,['Name','Salary']]
Specific rows	df.loc[[101,103,105]]
Specific rows & columns	df.loc[[101,103],['Name','Salary']]

---

8. Update Specific Values (iloc & loc)

Operation	Code
Update using iloc	df.iloc[0,1] = 500
Update using loc	df.loc[101,'Salary'] = 500
Update entire row (iloc)	df.iloc[0] = [101,'A',500,'Male']
Update entire row (loc)	df.loc[101] = [101,'A',500,'Male']
Update entire column	df['Salary'] = 1000
Update using condition	df.loc[df['Gender']=='Female','Salary'] = 800
Update multiple columns using condition	df.loc[df['Salary']<500,['Salary','Bonus']] = [500,100]

---

8. NEW COLUMN

Required Import
import pandas as pd
import numpy as np

Ways to Add Columns in Pandas

Operation	Code
1. New Column = Old Column × 4 + 2000	df['New_Salary'] = df['Salary'] * 4 + 2000
2. New Column = List	df['Department'] = ['HR','IT','Sales','Finance']
3. Add Column(s) from another DataFrame (concat)	df = pd.concat([df1, df2], axis=1)
4. New Column = def Function	def calc_bonus(x): return x * 0.10

	df['Bonus'] = df['Salary'].apply(calc_bonus)
5. New Column = apply() with Function	def square(x): return x ** 2 df['Square'] = df['Salary'].apply(square)
6. New Column = Lambda Function	tax = lambda x: x * 0.10 df['Tax'] = df['Salary'].apply(tax)
7. New Column = Excellent / Good / Average / Poor using np.select()	conditions = [df['Marks']>=90, df['Marks']>=75, df['Marks']>=50, df['Marks']<50]choices = ['Excellent','Good','Average','Poor'] df['Grade'] = np.select(conditions, choices)
8. New Column = Excellent / Good / Average / Poor using np.where()	df['Grade'] = np.where(df['Marks']>=90,'Excellent', np.where(df['Marks']>=75,'Good', np.where(df['Marks']>=50,'Average','Poor')))

9. DROP

Ways to Drop Columns / Rows in Pandas

Operation	Code
1. Drop Specific Column	df = df.drop('Salary', axis=1)
2. Drop Columns Having Any NULL Value	df = df.dropna(axis=1)
3. Drop Row if Any One Value is NULL	df = df.dropna()
4. Drop Row if Two or More Values are NULL	df = df.dropna(thresh=df.shape[1]-1)
5. Drop Row if Less Than 4 Non-NULL Values Exist	df = df.dropna(thresh=4)
6. Drop Row if Entire Row is NULL	df = df.dropna(how='all')
7. Drop Row if Gender = Female	df = df[df['Gender'] != 'Female']
8. Drop Row if Salary < 50000	df = df[df['Salary'] >= 50000]
9. Drop Row if Salary*2 + Bonus + 500 < 50000	df = df[(df['Salary']*2 + df['Bonus'] + 500) >= 50000]
10. Drop Row using def Function	def keep_row(x): return x >= 50000 df = df[df['Salary'].apply(keep_row)]
11. Drop Duplicate Employee IDs	df = df.drop_duplicates(subset='Emp_ID')
12. Drop Only if Emp_ID AND Gender Both Same	df = df.drop_duplicates(subset=['Emp_ID','Gender'])
13. Drop if Emp_ID Same OR Name Same (Keep First)	df = df.drop_duplicates(subset=['Emp_ID']) df = df.drop_duplicates(subset=['Name'])

Alternative for #13 (Keep Last)

df = df.drop_duplicates(subset=['Emp_ID'], keep='last')

df = df.drop_duplicates(subset=['Name'], keep='last')

Alternative for #13 (Remove All Duplicates)

df = df.drop_duplicates(subset=['Emp_ID'], keep=False)

df = df.drop_duplicates(subset=['Name'], keep=False)

Notes

Parameter	Meaning
axis=0	Rows

axis=1	Columns
how='any'	Drop if any NULL exists
how='all'	Drop only if all values are NULL
thresh=n	Keep rows with at least n non-NULL values
subset=	Columns checked for duplicates
keep='first'	Keep first occurrence (default)
keep='last'	Keep last occurrence
keep=False	Remove all duplicates

---

10. APPLY

Use this one example to understand **apply()** completely.

```
import pandas as pd

df = pd.DataFrame({
   'Emp_ID':[101,102,103,104,105],
   'Name':['Aman','Riya','Karan','Priya','Rohit'],
   'Salary':[40000,55000,70000,45000,90000],
   'Age':[22,25,30,24,35]
})

print(df)
```

Output

Emp_ID	Name	Salary	Age
101	Aman	40000	22
102	Riya	55000	25
103	Karan	70000	30
104	Priya	45000	24
105	Rohit	90000	35

---

1. apply() on One Column (using def)

```
def bonus(x):
   return x * 0.10

df['Bonus'] = df['Salary'].apply(bonus)
```

Result

Salary	Bonus
40000	4000
55000	5500
70000	7000
45000	4500
90000	9000

---

2. apply() on One Column (using Lambda Variable)

```
tax = lambda x: x * 0.05

df['Tax'] = df['Salary'].apply(tax)
```

Result

Salary	Tax
40000	2000
55000	2750
70000	3500
45000	2250

90000	4500

---

3. apply() on Multiple Columns (axis=1)

```
def total(row):
  return row['Salary'] + row['Age'] * 100

df['Total'] = df.apply(total, axis=1)
```

Result

Salary	Age	Total
40000	22	42200
55000	25	57500
70000	30	73000
45000	24	47400
90000	35	93500

---

4. apply() Returning Categories

```
def grade(x):
  if x >= 80000:
    return 'Excellent'
  elif x >= 60000:
    return 'Good'
  elif x >= 45000:
    return 'Average'
  else:
    return 'Poor'

df['Grade'] = df['Salary'].apply(grade)
```

Result

Salary	Grade
40000	Poor
55000	Average
70000	Good
45000	Average
90000	Excellent

---

5. apply() on Every Column (axis=0)

```
def add1000(x):
  return x + 1000

df[['Salary']] = df[['Salary']].apply(add1000)
```

Result

Salary
41000
56000
71000
46000
91000

---

11. PANDA CHARTS

Graph	Code
1. Line Plot	df.plot(x='Month', y='Sales', kind='line') # marker='o', linewidth=2, grid=True, title='Sales Trend'

2. Scatter Plot	df.plot(x='Age', y='Salary', kind='scatter') # alpha=0.7, figsize=(8,5), title='Age vs Salary'
3. Bar Chart (Vertical)	df.plot(x='Department', y='Sales', kind='bar') # rot=45, grid=True, title='Department Sales'
4. Bar Chart (Horizontal)	df.plot(x='Department', y='Sales', kind='barh') # figsize=(8,5), title='Department Sales'
5. Histogram	df['Salary'].plot(kind='hist', bins=10) # bins=20, density=True, title='Salary Distribution'
6. Box Plot	df[['Salary','Bonus']].plot(kind='box') # vert=True, figsize=(6,4), title='Salary & Bonus Boxplot'
7. Area Chart	df.plot(x='Month', y=['Sales','Profit'], kind='area', stacked=True) # alpha=0.7, title='Sales & Profit Area'
8. Pie Chart	df.set_index('Department')['Sales'].plot(kind='pie', autopct='%1.1f%%', startangle=90) # shadow=True, title='Sales Share'
9. Density (KDE) Plot	df['Salary'].plot(kind='kde') # linewidth=2, grid=True, title='Salary Density'
10. Hexbin Plot	df.plot(x='Age', y='Salary', kind='hexbin', gridsize=10) # gridsize=20, figsize=(8,5), title='Age vs Salary Hexbin'

12. MATPLOTLIB

Graph	Code
1. Line Plot	plt.plot(df['Month'], df['Sales']) # marker='o', linewidth=2, linestyle='--', color='blue', label='Sales', alpha=0.8
2. Scatter Plot	plt.scatter(df['Age'], df['Salary']) # s=100, alpha=0.7, marker='o', color='red', label='Employees'
3. Bar Chart (Vertical)	plt.bar(df['Department'], df['Sales']) # width=0.6, color='green', edgecolor='black', label='Sales'
4. Bar Chart (Horizontal)	plt.barh(df['Department'], df['Sales']) # height=0.6, color='orange', edgecolor='black'
5. Histogram	plt.hist(df['Salary'], bins=10) # bins=20, density=True, cumulative=False, edgecolor='black', alpha=0.7
6. Box Plot	plt.boxplot(df['Salary']) # vert=True, showmeans=True, showfliers=True, patch_artist=True
7. Area Chart	plt.fill_between(df['Month'], df['Sales']) # alpha=0.5, color='skyblue', label='Sales Area'
8. Pie Chart	plt.pie(df['Sales'], labels=df['Department']) # autopct='%1.1f%%', startangle=90, shadow=True, explode=[0.1,0,0,0]
9. Density (KDE) Plot	df['Salary'].plot(kind='kde') # linewidth=2, grid=True, title='Salary Density', figsize=(8,5)
10. Hexbin Plot	plt.hexbin(df['Age'], df['Salary'], gridsize=10) # gridsize=20, mincnt=1, cmap='viridis'
11. Stack Plot	plt.stackplot(df['Month'], df['Sales'], df['Profit']) # labels=['Sales','Profit'], alpha=0.7
12. Stem Plot	plt.stem(df['Month'], df['Sales']) # linefmt='-', markerfmt='o', basefmt='black'
13. Step Plot	plt.step(df['Month'], df['Sales']) # where='mid', linewidth=2, label='Sales'
14. Error Bar Plot	plt.errorbar(df['Month'], df['Sales'], yerr=df['Error']) # fmt='o', capsize=5, linewidth=2, label='Sales'
15. Event Plot	plt.eventplot(df['Sales']) # orientation='horizontal', linewidths=2, linelengths=0.8
16. Polar Plot	plt.polar(theta, r) # marker='o', linewidth=2, markersize=6

---

13. FUNCTION+LAMBDA

1. DEF FUNCTION (Basic)

```
def func1(variable1):
  return variable1 ** 2

print(func1(5))
print(func1(10))
```

---

2. DEF FUNCTION ON ELEMENTS OF A LIST

```
numbers = [10, 20, 30, 40, 50]

def func2(variable1):
  return variable1 + 10

result = list(map(func2, numbers))

print(result)
```

---

3. DEF FUNCTION TO DO OPERATION ON ELEMENTS OF LIST WHICH ARE > 50

```
numbers = [20, 45, 55, 70, 90]

def func3(variable1):
  if variable1 > 50:
    return variable1 * 2
  else:
    return variable1

result = list(map(func3, numbers))

print(result)
```

---

4. DEF FUNCTION TO DO OPERATION ON ELEMENTS OF LIST

If > 50 → x × 2

Else → x × 3

```
numbers = [20, 45, 55, 70, 90]

def func4(variable1):
  if variable1 > 50:
    return variable1 * 2
  else:
    return variable1 * 3

result = list(map(func4, numbers))

print(result)
```

---

5. DEF FUNCTION + APPLY (Salary & Age)

```
import pandas as pd

df = pd.DataFrame({
  'Name':['A','B','C','D'],
  'Salary':[40000,65000,70000,30000],
  'Age':[22,30,28,21]
})

def func5(variable1):
  if variable1['Age'] >= 25:
    return variable1['Salary'] * 0.20
```

```
    else:
        return variable1['Salary'] * 0.10

df['Bonus'] = df.apply(func5, axis=1)

print(df)
```

---

6. LAMBDA FUNCTION (Basic)

```
func6 = lambda variable1: variable1 ** 2

print(func6(5))
print(func6(10))
```

---

7. LAMBDA FUNCTION ON ELEMENTS OF A LIST

```
numbers = [10, 20, 30, 40, 50]

result = list(map(lambda variable1: variable1 + 10, numbers))

print(result)
```

---

8. LAMBDA FUNCTION TO DO OPERATION ON ELEMENTS OF LIST WHICH ARE > 50

```
numbers = [20, 45, 55, 70, 90]

result = list(map(lambda variable1: variable1 * 2 if variable1 > 50 else variable1,
numbers))

print(result)
```

---

9. LAMBDA FUNCTION TO DO OPERATION ON ELEMENTS OF LIST

If > 50 → x × 2

Else → x × 3

```
numbers = [20, 45, 55, 70, 90]

result = list(map(lambda variable1: variable1 * 2 if variable1 > 50 else variable1 * 3,
numbers))

print(result)
```

---

10. LAMBDA FUNCTION + APPLY (Salary & Age)

```
import pandas as pd

df = pd.DataFrame({
    'Name':['A','B','C','D'],
    'Salary':[40000,65000,70000,30000],
    'Age':[22,30,28,21]
})

df['Bonus'] = df.apply(
    lambda variable1: variable1['Salary'] * 0.20 if variable1['Age'] >= 25 else
variable1['Salary'] * 0.10,
    axis=1
)

print(df)
```

This keeps the same structure as before, with:

- Function names: func1, func2, ..., func6
- Lambda parameter: variable1

- apply() parameter: variable1 instead of row
- No other changes to the logic.

---

14. LAMBDA, MAP, FILTER

1. map() ONLY

Syntax

map(function, iterable)

Example 1

```
numbers = [10, 20, 30, 40, 50]

def func1(variable1):
  return variable1 + 5

result = list(map(func1, numbers))

print(result)
```

Output

[15, 25, 35, 45, 55]

Example 2

```
numbers = [1, 2, 3, 4, 5]

def func2(variable1):
  return variable1 ** 2

result = list(map(func2, numbers))

print(result)
```

---

2. filter() ONLY

Syntax

filter(function, iterable)

Example 1

```
numbers = [10, 25, 40, 55, 70]

def func3(variable1):
  return variable1 > 40

result = list(filter(func3, numbers))

print(result)
```

Output

[55, 70]

Example 2

```
numbers = [11, 12, 13, 14, 15]

def func4(variable1):
  return variable1 % 2 == 0

result = list(filter(func4, numbers))
```

```
print(result)
```

---

3. lambda ONLY

Syntax

lambda arguments : expression

Example 1

```
func5 = lambda variable1: variable1 * 10

print(func5(5))
```

Example 2

```
func6 = lambda variable1, variable2: variable1 + variable2

print(func6(10, 20))
```

---

4. map() + lambda

Example 1

```
numbers = [10, 20, 30, 40]

result = list(map(lambda variable1: variable1 * 2, numbers))

print(result)
```

Example 2

```
numbers = [2, 4, 6, 8]

result = list(map(lambda variable1: variable1 ** 2, numbers))

print(result)
```

---

5. filter() + lambda

Example 1

```
numbers = [10, 25, 40, 55, 70]

result = list(filter(lambda variable1: variable1 > 40, numbers))

print(result)
```

Example 2

```
numbers = [11, 12, 13, 14, 15]

result = list(filter(lambda variable1: variable1 % 2 == 0, numbers))

print(result)
```

---

6. filter() + map()

(Filter first, then perform operation)

```
numbers = [10, 25, 40, 55, 70]

def func7(variable1):
  return variable1 > 40

def func8(variable1):
  return variable1 * 2

result = list(map(func8, filter(func7, numbers)))
```

```
print(result)
```

Output

```
[110, 140]
```

---

7. map() + filter() + lambda

```
numbers = [10, 25, 40, 55, 70]

result = list(
  map(
    lambda variable1: variable1 * 2,
    filter(
      lambda variable1: variable1 > 40,
      numbers
    )
  )
)

print(result)
```

---

8. map() + filter() + def

```
numbers = [10, 25, 40, 55, 70]

def func9(variable1):
  return variable1 > 40

def func10(variable1):
  return variable1 * 2

result = list(map(func10, filter(func9, numbers)))

print(result)
```

---

9. map() WITH TWO LISTS

```
list1 = [10, 20, 30]
list2 = [1, 2, 3]

def func11(variable1, variable2):
  return variable1 + variable2

result = list(map(func11, list1, list2))

print(result)
```

Output

```
[11, 22, 33]
```

---

10. map() WITH TWO LISTS + lambda

```
list1 = [10, 20, 30]
list2 = [1, 2, 3]

result = list(map(lambda variable1, variable2: variable1 + variable2, list1, list2))

print(result)
```

---

11. filter() + map() + Conditional Operation

Numbers > 50 → ×2

Numbers ≤ 50 are removed by filter()

```
numbers = [20, 45, 55, 70, 90]

result = list(
  map(
    lambda variable1: variable1 * 2,
    filter(
      lambda variable1: variable1 > 50,
      numbers
    )
  )
)

print(result)
```

Output

[110, 140, 180]

---

12. filter() + map() Using def

```
numbers = [20, 45, 55, 70, 90]

def func12(variable1):
  return variable1 > 50

def func13(variable1):
  return variable1 * 2

result = list(map(func13, filter(func12, numbers)))
print(result)
```

Quick Summary

Function	Purpose	Example
map()	Apply a function to every element	map(func, list)
filter()	Keep only elements satisfying a condition	filter(func, list)
lambda	Create an anonymous (inline) function	lambda x: x*2
map() + lambda	Apply an inline function to each element	map(lambda x: x+10, list)
filter() + lambda	Filter using an inline condition	filter(lambda x: x>50, list)
filter() + map()	Filter first, then transform	map(func2, filter(func1, list))
map() with multiple lists	Process elements from multiple iterables together	map(func, list1, list2)

These examples cover the most common interview and practical use cases for map, filter, and lambda, both separately and in combination.

---

15. DATA STRUCTURES

1. LIST

Definition: Ordered, mutable collection that allows duplicates.

Properties: Ordered ✅ | Mutable ✅ | Duplicates ✅ | Indexed ✅

No.	Code	Purpose
1	list1=[10,20,30]	Create list

No.	Code	Purpose
2	print(list1[1])	Access element
3	list1.append(40)	Add at end
4	list1.insert(1,15)	Insert element
5	list1.remove(20)	Remove value
6	list1.pop()	Remove last element
7	print(len(list1))	Length
8	print(max(list1))	Maximum
9	print(min(list1))	Minimum
10	print(sum(list1))	Sum

---

2. TUPLE

Definition: Ordered, immutable collection that allows duplicates.

Properties: Ordered ✅ | Immutable ✅ | Duplicates ✅ | Indexed ✅

No.	Code	Purpose
1	tuple1=(10,20,30)	Create tuple
2	print(tuple1[0])	Access element
3	print(len(tuple1))	Length
4	print(max(tuple1))	Maximum
5	print(min(tuple1))	Minimum
6	print(sum(tuple1))	Sum
7	print(tuple1.count(20))	Count value
8	print(tuple1.index(30))	Index of value
9	tuple2=tuple1+(40,)	Add element (new tuple)
10	print(20 in tuple1)	Membership

---

3. SET

Definition: Unordered, mutable collection of unique values.

Properties: Unordered ✅ | Mutable ✅ | Unique Only ✅ | Indexed ❌

No.	Code	Purpose
1	set1={10,20,30}	Create set
2	set1.add(40)	Add element
3	set1.remove(20)	Remove element
4	set1.discard(50)	Remove safely
5	set1.pop()	Remove random element
6	print(len(set1))	Length
7	print(max(set1))	Maximum
8	print(min(set1))	Minimum
9	print(20 in set1)	Membership
10	set1.clear()	Remove all

---

4. DICTIONARY

Definition: Collection of **key-value pairs**.

Properties: Ordered ✅ | Mutable ✅ | Keys Unique ✅ | Indexed by Keys ✅

No.	Code	Purpose
1	dict1={'A':10,'B':20}	Create dictionary
2	print(dict1['A'])	Access value
3	dict1['C']=30	Add key-value
4	dict1['A']=100	Update value
5	dict1.pop('B')	Remove key

No.	Code	Purpose
6	print(dict1.keys())	Keys
7	print(dict1.values())	Values
8	print(dict1.items())	Key-value pairs
9	print(len(dict1))	Length
10	dict1.clear()	Remove all

---

5. STRING

Definition: Ordered, immutable sequence of characters.

Properties: Ordered ✅ | Immutable ✅ | Indexed ✅ | Duplicates ✅

No.	Code	Purpose
1	string1="Python"	Create string
2	print(string1[0])	Access character
3	print(len(string1))	Length
4	print(string1.upper())	Uppercase
5	print(string1.lower())	Lowercase
6	print(string1.replace("P","J"))	Replace
7	print(string1.split("t"))	Split
8	print(string1.find("h"))	Find index
9	print(string1.startswith("Py"))	Starts with
10	print(string1.endswith("on"))	Ends with

CONVERSIONS

From	To	Code
List → Tuple	tuple(list1)	
Tuple → List	list(tuple1)	
List → Set	set(list1)	
Set → List	list(set1)	
Tuple → Set	set(tuple1)	
Set → Tuple	tuple(set1)	
String → List	list(string1)	
List → String	"".join(list1) *(list must contain strings)*	
String → Tuple	tuple(string1)	
String → Set	set(string1)	
Dictionary → List	list(dict1) *(keys)*	
Dictionary → Tuple	tuple(dict1) *(keys)*	
Dictionary → Set	set(dict1) *(keys)*	

LIST COMPREHENSION

Basic Format

[expression for variable1 in iterable]

Example

[variable1*2 for variable1 in range(5)]

With if

[expression for variable1 in iterable if condition]

Example

[variable1 for variable1 in range(10) if variable1>5]

With if...else

[true_value if condition else false_value for variable1 in iterable]

Example

[variable1*2 if variable1>5 else variable1*3 for variable1 in range(10)]

Nested Loop

[(variable1,variable2) for variable1 in range(3) for variable2 in range(3)]

Multiple if

[variable1 for variable1 in range(20) if variable1>5 if variable1<15]

Dictionary Comprehension

{variable1:variable1**2 for variable1 in range(5)}

Set Comprehension

{variable1**2 for variable1 in range(5)}

Generator Expression

(variable1**2 for variable1 in range(5))

These examples cover the most common operations and syntax you'll need for coding practice and interviews.
